

EXERCÍCIO: IMPLEMENTAÇÃO E ANÁLISE DE ALGORITMOS RECURSIVOS E ITERATIVOS

Nome: Matheus de Souza Rodrigues

Matricula: UC24101565

Curso: Ciência da Computação

Disciplina: *Análise de algoritmos*

Professor: ZOE ROBERTO MAGALHAES JUNIOR

Universidade: Universidade Católica de Brasília

Data: 08/09/2026

1. INTRODUÇÃO

Este trabalho apresenta a implementação e análise de algoritmos nas versões iterativa e recursiva. Foram desenvolvidas soluções para três problemas: contagem de zeros em uma matriz, inversão de uma string e identificação da maior ocorrência de uma mesma letra em uma string.

Para cada algoritmo foram analisadas as complexidades temporal e espacial, considerando a notação Big O. Também foram realizados experimentos computacionais, com 32 execuções para cada tamanho de entrada, a fim de comparar os resultados empíricos com as previsões teóricas.

Os experimentos registraram o tempo médio de execução. Para a análise do consumo de pilha, foi considerada a profundidade máxima estimada das chamadas recursivas, permitindo avaliar o comportamento dos algoritmos conforme o tamanho da entrada aumenta.

2. IMPLEMENTAÇÕES ITERATIVAS

2.1 Problema 1: Contador de zeros em uma matriz

O objetivo deste algoritmo é determinar a quantidade de elementos com valor igual a zero presentes em uma matriz. Na implementação iterativa, a matriz é percorrida utilizando dois laços de repetição: o primeiro percorre as linhas e o segundo percorre as colunas. Para cada elemento encontrado, é realizada uma verificação e, caso seu valor seja zero, um contador é incrementado.

O código-fonte completo da implementação está disponível nos arquivos em C entregues junto com este relatório, conforme listado no Anexo A.

2.1.1 Complexidade temporal

Considerando uma matriz de dimensão $N \times N$, o algoritmo percorre todos os seus elementos. Como existem N^2 elementos e cada elemento é analisado uma única vez, o número de operações cresce proporcionalmente a N^2 .

Assim, a complexidade temporal do algoritmo é:

$$T(N) = O(N^2)$$

2.1.2 Demonstração por indução

Para demonstrar a complexidade temporal por indução, considere $T(N)$ como o número de elementos percorridos pelo algoritmo em uma matriz $N \times N$.

Caso base:

Para $N = 1$, a matriz possui apenas um elemento. Portanto:

$$T(1) = 1$$

Hipótese de indução:

Suponha que, para uma matriz $N \times N$, o algoritmo realize N^2 operações para verificar todos os elementos.

Passo indutivo:

Ao aumentar a dimensão da matriz de N para $N + 1$, o número total de elementos passa a ser:

$$T(N + 1) = (N + 1)^2$$

Desenvolvendo:

$$T(N + 1) = N^2 + 2N + 1$$

Portanto, o algoritmo continua realizando uma operação para cada elemento da matriz. Assim, por indução, conclui-se que:

$$T(N) = O(N^2)$$

2.1.3 Complexidade espacial

A implementação iterativa utiliza apenas variáveis auxiliares para controlar o percurso da matriz e armazenar a quantidade de zeros encontrada. Não são realizadas chamadas recursivas.

Dessa forma, a quantidade de memória adicional utilizada na pilha permanece constante, independentemente do tamanho da matriz.

Portanto, a complexidade espacial da implementação iterativa, considerando especificamente o consumo da pilha, é:

$$S(N) = O(1)$$

2.1.4 Demonstração da complexidade espacial

A complexidade espacial considera especificamente o consumo da pilha de execução.

Caso base:

Para $N = 1$, o algoritmo utiliza apenas uma quantidade constante de memória para armazenar as variáveis auxiliares. Portanto:

$$S(1) = C$$

onde C representa uma quantidade constante de memória.

Hipótese de indução:

Suponha que, para uma matriz de tamanho $N \times N$, a quantidade de memória adicional utilizada na pilha seja constante.

Passo indutivo:

Ao aumentar o tamanho da matriz para $(N + 1) \times (N + 1)$, o algoritmo continua utilizando apenas as mesmas variáveis auxiliares. Os laços de repetição não criam novas chamadas de função na pilha.

Assim:

$$S(N + 1) = C$$

Portanto, por indução, a quantidade de memória utilizada na pilha permanece constante:

$$S(N) = O(1)$$

2.2 Problema 2: Inversor de String

O objetivo deste algoritmo é inverter uma string, fazendo com que o primeiro caractere ocupe a última posição, o segundo ocupe a penúltima posição e assim sucessivamente.

Na implementação iterativa, a string é convertida para um vetor de caracteres. Em seguida, são realizadas trocas entre os caracteres das extremidades, avançando em direção ao centro da string. Dessa forma, cada caractere é processado no máximo uma vez.

2.2.1 Complexidade temporal

Considere uma string de tamanho N . O algoritmo percorre metade dos caracteres da string, realizando uma troca em cada iteração.

Assim, o número de operações é proporcional a $N/2$. Como constantes são desconsideradas na notação Big O:

$$T(N) = O(N)$$

Portanto, a complexidade temporal da implementação iterativa é $O(N)$.

2.2.2 Complexidade espacial

A implementação utiliza algumas variáveis auxiliares, como o índice do laço e a variável temporária utilizada durante as trocas.

Não são realizadas chamadas recursivas. Portanto, considerando especificamente o consumo da pilha, a quantidade de memória adicional permanece constante.

Assim:

$$S(N) = O(1)$$

Portanto, a complexidade espacial da implementação iterativa é $O(1)$.

2.2.3 Demonstração da complexidade temporal

Considere $T(N)$ como o número de operações realizadas para inverter uma string de tamanho N .

Caso base:

Para $N = 1$, não é necessária nenhuma troca, portanto o número de operações é constante.

Hipótese de indução:

Suponha que uma string de tamanho N necessite de uma quantidade de operações proporcional a N .

Passo indutivo:

Para uma string de tamanho $N + 1$, o algoritmo continua percorrendo os caracteres até o centro da string. O número de iterações cresce proporcionalmente ao tamanho da entrada.

Como a constante $1/2$ é desconsiderada na notação Big O:

$$T(N + 1) = O(N + 1) = O(N)$$

Portanto, por indução, conclui-se que:

$$T(N) = O(N)$$

2.2.4 Demonstração da complexidade espacial

Considere $S(N)$ como o consumo adicional da pilha durante a execução.

Caso base:

Para $N = 1$, são utilizadas apenas as variáveis auxiliares do método, resultando em uma quantidade constante de memória.

Hipótese de indução:

Suponha que para uma string de tamanho N o algoritmo utilize uma quantidade constante de memória na pilha.

Passo indutivo:

Ao aumentar a string para $N + 1$ caracteres, o algoritmo continua utilizando as mesmas variáveis auxiliares e não realiza novas chamadas de função.

Portanto:

$$S(N + 1) = C$$

onde C é uma constante.

Assim, por indução:

$$S(N) = O(1)$$

2.3 Problema 3: Maior ocorrência de uma mesma letra

O objetivo deste algoritmo é identificar quantas vezes a letra que mais se repete aparece em uma string.

Na implementação iterativa, é utilizado um vetor de frequência para armazenar a quantidade de ocorrências de cada caractere. Primeiro, a string é percorrida e a frequência de cada caractere é contabilizada. Em seguida, o vetor de frequência é percorrido para identificar o maior valor encontrado.

2.3.1 Complexidade temporal

Considere uma string de tamanho N . O primeiro laço percorre todos os N caracteres da string. Depois, o vetor de frequência é percorrido.

Como o vetor possui tamanho fixo de 256 posições, seu custo é constante. Portanto:

$$T(N) = O(N + 256)$$

Como constantes são desconsideradas na notação Big O:

$$T(N) = O(N)$$

Assim, a complexidade temporal da implementação iterativa é $O(N)$.

2.3.2 Complexidade espacial

A implementação utiliza um vetor de frequência com tamanho fixo de 256 posições, além de algumas variáveis auxiliares.

Considerando especificamente o consumo da pilha, não são realizadas chamadas recursivas. Dessa forma, a quantidade de memória utilizada na pilha permanece constante em relação ao tamanho da entrada.

Portanto:

$$S(N) = O(1)$$

O vetor de frequência ocupa memória adicional, mas seu tamanho é fixo e não depende de N .

2.3.3 Demonstração da complexidade temporal por indução

Considere $T(N)$ como o número de operações necessárias para percorrer uma string de tamanho N .

Caso base: Para $N = 1$, apenas um caractere precisa ser processado, resultando em uma quantidade constante de operações.

Hipótese de indução: Suponha que, para uma string de tamanho N , o algoritmo realize uma quantidade de operações proporcional a N .

Passo indutivo: Ao aumentar a string para $N + 1$ caracteres, o algoritmo realiza uma operação adicional para processar o novo caractere. Assim:

$$T(N + 1) = T(N) + C$$

onde C representa uma quantidade constante de operações.

Portanto, o crescimento continua sendo linear. Assim, por indução:

$$T(N) = O(N)$$

2.3.4 Demonstração da complexidade espacial por indução

Considere $S(N)$ como o consumo adicional da pilha durante a execução.

Caso base: Para $N = 1$, são utilizadas apenas as variáveis auxiliares do método, resultando em uma quantidade constante de memória.

Hipótese de indução: Suponha que, para uma string de tamanho N , o algoritmo utilize uma quantidade constante de memória na pilha.

Passo indutivo: Ao aumentar a entrada para $N + 1$ caracteres, o algoritmo continua utilizando as mesmas variáveis auxiliares e não realiza chamadas recursivas.

Assim:

$$S(N + 1) = C$$

onde C é uma constante.

Portanto, por indução:

$$S(N) = O(1)$$

3. IMPLEMENTAÇÕES RECURSIVAS

3.1 Problema 1: Contador de zeros em uma matriz

Na implementação recursiva, a matriz é percorrida elemento por elemento por meio de chamadas sucessivas da função. A cada chamada, o algoritmo verifica se o elemento atual é zero e, em seguida, realiza uma nova chamada para o próximo elemento.

Quando todos os elementos da matriz já foram processados, ocorre o caso base, encerrando a recursão.

O código-fonte completo da implementação está disponível nos arquivos em C entregues junto com este relatório, conforme listado no Anexo A.

3.1.1 Complexidade temporal

Considerando uma matriz $N \times N$, existem N^2 elementos a serem analisados.

A cada chamada recursiva, apenas um elemento é processado. Portanto, são realizadas aproximadamente N^2 chamadas até que todos os elementos sejam percorridos.

Assim:

$$T(N) = O(N^2)$$

Portanto, a complexidade temporal da implementação recursiva é $O(N^2)$.

3.1.2 Demonstração da complexidade temporal por indução

Considere $T(N)$ como o número de elementos processados em uma matriz $N \times N$.

Caso base: Para $N = 1$, existe apenas um elemento a ser analisado. Portanto, $T(1)$ é constante.

Hipótese de indução: Suponha que uma matriz $N \times N$ necessite de N^2 chamadas para processar todos os seus elementos.

Passo indutivo: Para uma matriz de dimensão $N + 1$, o número de elementos será:

$$T(N + 1) = (N + 1)^2$$

Desenvolvendo:

$$T(N + 1) = N^2 + 2N + 1$$

Portanto, o número de chamadas cresce proporcionalmente ao quadrado do tamanho da matriz.

Assim, por indução:

$$T(N) = O(N^2)$$

3.1.3 Complexidade espacial

Diferentemente da implementação iterativa, a versão recursiva realiza uma nova chamada da função para cada elemento da matriz.

Como existem N^2 elementos, podem existir até N^2 chamadas recursivas simultaneamente na pilha antes que a execução retorne.

Portanto, considerando especificamente o consumo da pilha:

$$S(N) = O(N^2)$$

3.1.4 Demonstração da complexidade espacial por indução

Considere $S(N)$ como a profundidade máxima da pilha durante a execução.

Caso base: Para $N = 1$, existe apenas uma chamada recursiva, resultando em uma quantidade constante de memória.

Hipótese de indução: Suponha que, para uma matriz $N \times N$, a profundidade máxima da pilha seja proporcional a N^2 .

Passo indutivo: Para uma matriz de dimensão $N + 1$, existem:

$$(N + 1)^2$$

elementos que precisam ser processados. Como cada elemento gera uma nova chamada recursiva antes do retorno das chamadas anteriores, a profundidade máxima também cresce proporcionalmente ao número de elementos.

Assim:

$$S(N + 1) = (N + 1)^2$$

Portanto:

$$S(N) = O(N^2)$$

3.2 Problema 2: Inversor de String

Na implementação recursiva, a string é convertida para um vetor de caracteres. A função realiza a troca entre o primeiro e o último caractere e, em seguida, chama a si mesma passando os índices que avançam em direção ao centro da string.

O processo continua até que os índices se encontrem ou ultrapassem a posição central, caracterizando o caso base.

O código-fonte completo da implementação está disponível nos arquivos em C entregues junto com este relatório, conforme listado no Anexo A.

3.2.1 Complexidade temporal

Considere uma string de tamanho N .

A cada chamada recursiva, dois caracteres são trocados e os índices avançam em direção ao centro. Portanto, aproximadamente $N/2$ chamadas são realizadas.

Assim:

$$T(N) = O(N/2)$$

Desconsiderando constantes na notação Big O:

$$T(N) = O(N)$$

Portanto, a complexidade temporal da implementação recursiva é $O(N)$.

3.2.2 Complexidade espacial

Cada chamada recursiva permanece armazenada na pilha até que o caso base seja atingido.

Como aproximadamente $N/2$ chamadas são realizadas, a profundidade máxima da pilha é proporcional a N .

Assim:

$$S(N) = O(N)$$

Portanto, a complexidade espacial, considerando especificamente o consumo da pilha, é $O(N)$.

3.2.3 Demonstração da complexidade temporal por indução

Considere $T(N)$ como o número de operações necessárias para inverter uma string de tamanho N .

Caso base: Para $N = 1$, nenhuma troca é necessária. Portanto, o número de operações é constante.

Hipótese de indução: Suponha que uma string de tamanho N necessite de uma quantidade de operações proporcional a N .

Passo indutivo: Para uma string de tamanho $N + 1$, a função realiza uma troca entre as extremidades e chama novamente a si mesma para processar os caracteres restantes.

Dessa forma:

$$T(N + 1) = T(N) + C$$

onde C representa uma quantidade constante de operações.

Logo, o crescimento é linear:

$$T(N) = O(N)$$

3.2.4 Demonstração da complexidade espacial por indução

Considere $S(N)$ como a profundidade máxima da pilha.

Caso base: Para $N = 1$, não são necessárias novas chamadas recursivas, resultando em uma quantidade constante de memória.

Hipótese de indução: Suponha que uma string de tamanho N apresente profundidade de pilha proporcional a N .

Passo indutivo: Para uma string de tamanho $N + 1$, uma nova chamada recursiva é adicionada antes que a chamada anterior seja finalizada.

Assim:

$$S(N + 1) = S(N) + C$$

Portanto, a profundidade da pilha cresce linearmente com o tamanho da entrada.

Assim:

$$S(N) = O(N)$$

3.3 Problema 3: Maior ocorrência de uma mesma letra

Na implementação recursiva, o algoritmo percorre a string caractere por caractere. Para cada caractere encontrado, é realizada uma segunda função recursiva responsável por contar quantas vezes aquele caractere aparece em toda a string.

Após obter a quantidade de ocorrências, o algoritmo compara esse valor com a maior frequência encontrada até o momento. Em seguida, realiza uma nova chamada recursiva para analisar o próximo caractere.

Dessa forma, existe uma recursão externa responsável por percorrer a string e uma recursão interna responsável por contar as ocorrências de cada caractere.

O código-fonte completo da implementação está disponível nos arquivos em C entregues junto com este relatório, conforme listado no Anexo A.

3.3.1 Complexidade temporal

Considere uma string de tamanho N .

A função `maiorOcorrenciaRecursivo` percorre os N caracteres. Para cada caractere, a função `contarCaracter` percorre novamente toda a string, realizando N operações.

Assim, o número total de operações é aproximadamente:

$$T(N) = N \times N$$

Portanto:

$$T(N) = O(N^2)$$

A complexidade temporal da implementação recursiva é, portanto, $O(N^2)$.

3.3.2 Complexidade espacial

A função externa possui chamadas recursivas proporcionais a N .

Durante a execução de `contarCaracter`, também são realizadas chamadas recursivas. Entretanto, a recursão interna é executada enquanto uma chamada da função externa permanece na pilha.

A profundidade máxima continua sendo proporcional a N . Portanto:

$$S(N) = O(N)$$

Assim, considerando especificamente o consumo da pilha, a complexidade espacial é $O(N)$.

3.3.3 Demonstração da complexidade temporal por indução

Considere $T(N)$ como o número de operações realizadas para analisar uma string de tamanho N .

Caso base: Para $N = 1$, a função externa realiza uma chamada para `contarCaracter`, que percorre uma quantidade constante de elementos. Portanto, o custo é constante.

Hipótese de indução: Suponha que, para uma string de tamanho N , o algoritmo realize uma quantidade de operações proporcional a N^2 .

Passo indutivo: Para uma string de tamanho $N + 1$, a função externa processa mais um caractere e realiza uma nova chamada de `contarCaracter`, que percorre novamente a string. Dessa forma, o crescimento do custo adicional é proporcional a N .

Assim:

$$T(N + 1) = T(N) + O(N)$$

Como o custo acumulado cresce quadraticamente:

$$T(N) = O(N^2)$$

Portanto, por indução, conclui-se que a complexidade temporal é $O(N^2)$.

3.3.4 Demonstração da complexidade espacial por indução

Considere $S(N)$ como a profundidade máxima da pilha durante a execução.

Caso base: Para $N = 1$, existe apenas uma quantidade constante de chamadas, resultando em consumo constante da pilha.

Hipótese de indução: Suponha que, para uma string de tamanho N , a profundidade máxima da pilha seja proporcional a N .

Passo indutivo: Ao aumentar a entrada para $N + 1$, a recursão continua adicionando chamadas até percorrer a string. A profundidade da pilha cresce linearmente com o tamanho da entrada.

Assim:

$$S(N + 1) = S(N) + C$$

onde C representa uma quantidade constante de memória por chamada.

Portanto:

$$S(N) = O(N)$$

4. RESULTADOS EXPERIMENTAIS

Para os experimentos, o tamanho N representa a dimensão da matriz no Problema 1 e o número de caracteres da string nos Problemas 2 e 3. Cada combinação de problema, implementação e tamanho de entrada foi executada 32 vezes. O tempo foi medido em nanossegundos com um contador de alta resolução do Windows, e foi calculada a média das 32 execuções. A geração das entradas ocorreu antes da medição, para evitar que o custo de preparação dos dados contaminasse o tempo do algoritmo. No Problema 2, uma cópia da string foi preparada antes de cada medição para que a alteração provocada pela inversão não afetasse as execuções seguintes.

4.1 Problema 1: Contador de zeros em uma matriz

Para o Problema 1, foram utilizadas matrizes quadradas com $N = 10, 20, 30, 40$ e 50 . Os resultados abaixo correspondem à média de 32 execuções para cada tamanho.

N	Iterativo (ns)	Recursivo (ns)	Pilha iterativo	Pilha recursivo
10	212.50	587.50	1	100
20	1000.00	2584.38	1	400
30	2365.62	6175.00	1	900
40	4246.88	11312.50	1	1.600
50	4406.25	17493.75	1	2.500

O tempo médio aumentou de 212,50 ns para 4.406,25 ns na versão iterativa, enquanto a versão recursiva aumentou de 587,50 ns para 17.493,75 ns. Embora os valores individuais não formem uma curva perfeitamente regular, o crescimento é compatível com $O(N^2)$, pois o número de elementos processados cresce com o quadrado de N . A diferença entre as versões também se amplia: em $N = 50$, a recursiva apresentou aproximadamente quatro vezes o tempo da iterativa.

A profundidade máxima da pilha iterativa permaneceu em 1 em todos os tamanhos, enquanto a recursiva passou de 100 para 2.500 chamadas, exatamente acompanhando N^2 . Isso mostra que, neste algoritmo, a principal consequência da recursão não é alterar a ordem assintótica do tempo, mas aumentar fortemente o consumo de pilha.

4.2 Problema 2: Inversor de String

Para o Problema 2, foram utilizadas strings com $N = 100, 500, 1000, 1500$ e 2000 . Os resultados abaixo correspondem à média de 32 execuções para cada tamanho.

N	Iterativo (ns)	Recursivo (ns)	Pilha iterativo	Pilha recursivo
100	156.25	356.25	1	50
500	1096.88	1875.00	1	250

N	Iterativo (ns)	Recursivo (ns)	Pilha iterativo	Pilha recursivo
1000	1275.00	3900.00	1	500
1500	1887.50	5687.50	1	750
2000	2503.12	9396.88	1	1.000

Os tempos cresceram de 156,25 ns para 2.503,12 ns na versão iterativa e de 356,25 ns para 9.396,88 ns na recursiva. Há pequenas irregularidades entre alguns tamanhos, esperadas em medições de programas muito rápidos. Ainda assim, a tendência geral de crescimento é compatível com $O(N)$, pois cada chamada ou iteração trata uma quantidade proporcional de caracteres. As diferenças de tempo observadas não devem ser interpretadas como mudanças na complexidade assintótica.

A pilha iterativa permaneceu constante em 1. Na versão recursiva, a profundidade passou de 50 para 1.000 chamadas, correspondendo aproximadamente a $N/2$. Portanto, o consumo da pilha cresce linearmente com a entrada, como previsto por $O(N)$.

A implementação iterativa mantém consumo constante de pilha, enquanto a implementação recursiva apresenta crescimento linear da profundidade estimada da pilha, atingindo 1.000 chamadas para $N = 2.000$.

4.3 Problema 3: Maior ocorrência de uma mesma letra

Para o Problema 3, foram utilizadas strings com $N = 100, 500, 1000, 1500$ e 2000 . Os resultados abaixo correspondem à média de 32 execuções para cada tamanho.

N	Iterativo (ns)	Recursivo (ns)	Pilha iterativo	Pilha recursivo
100	740.62	53953.12	1	100
500	1512.50	1357878.12	1	500
1000	2421.88	7208406.25	1	1.000
1500	3309.38	13247509.38	1	1.500
2000	4218.75	22184193.75	1	2.000

A versão iterativa aumentou de 740,62 ns para 4.218,75 ns, enquanto a recursiva passou de 53.953,12 ns para 22.184.193,75 ns. O crescimento da versão iterativa permanece aproximadamente linear, enquanto o da recursiva se torna muito mais acentuado conforme N aumenta.

Na versão recursiva, para cada caractere a função de contagem percorre novamente a string. Assim, o trabalho é aproximadamente $N \times N$, o que explica o crescimento

quadrático. O efeito é visível nos dados: entre $N = 100$ e $N = 2.000$, o tempo recursivo aumenta aproximadamente 411 vezes, enquanto o iterativo aumenta cerca de 5,7 vezes. Isso evidencia o impacto prático da diferença entre $O(N)$ e $O(N^2)$.

A profundidade da pilha recursiva passou de 100 para 2.000 chamadas, acompanhando linearmente o tamanho da entrada. Portanto, apesar do tempo ser $O(N^2)$, o consumo máximo da pilha permanece $O(N)$.

5. COMPARAÇÃO ENTRE TEORIA E PRÁTICA

A comparação deve considerar tanto a tendência de crescimento quanto o custo de memória. Os tempos não precisam coincidir exatamente com as funções teóricas, pois Big O descreve a ordem de crescimento e não o valor absoluto do tempo. As 32 repetições por tamanho reduzem o efeito de uma execução isolada, mas não eliminam a interferência do ambiente.

5.1 Problema 1

No Problema 1, ambas as versões possuem tempo $O(N^2)$, pois percorrem os N^2 elementos da matriz. A diferença prática está principalmente na pilha: a iterativa mantém profundidade constante, enquanto a recursiva chega a N^2 chamadas.

Os dados confirmam essa tendência. Na versão iterativa, o tempo passou de 212,50 ns em $N = 10$ para 4.406,25 ns em $N = 50$. Na recursiva, passou de 587,50 ns para 17.493,75 ns. O aumento do custo recursivo acompanha o maior número de chamadas necessárias para percorrer todos os elementos.

A pilha fornece a evidência mais direta: a versão iterativa permaneceu em 1, enquanto a recursiva passou de 100 para 2.500 chamadas. Como $2.500 = 50^2$, o resultado empírico acompanha exatamente a ordem $O(N^2)$ prevista.

Assim, os resultados confirmam a principal diferença teórica entre as duas abordagens: a recursão aumenta significativamente o consumo da pilha.

5.2 Problema 2

No Problema 2, ambas as versões possuem tempo $O(N)$, porque realizam aproximadamente $N/2$ trocas. A versão iterativa usa pilha constante, enquanto a recursiva acumula chamadas até atingir aproximadamente $N/2$ níveis.

Os tempos apresentam alguma variação, mas a tendência global é crescente. Em $N = 2.000$, a versão iterativa registrou 2.503,12 ns e a recursiva 9.396,88 ns. As oscilações são compatíveis com interferências do ambiente de execução e com o fato de serem operações muito rápidas. Elas não contradizem a análise assintótica.

A profundidade medida confirma a previsão espacial: 1 chamada na versão iterativa e 1.000 chamadas na recursiva para $N = 2.000$, correspondendo a aproximadamente $N/2$.

5.3 Problema 3

O Problema 3 apresenta a maior diferença entre as implementações.

A versão iterativa possui complexidade temporal $O(N)$, pois percorre a string e utiliza um vetor de frequência de tamanho fixo.

A versão recursiva possui complexidade temporal $O(N^2)$, pois para cada caractere da string realiza uma nova contagem recursiva percorrendo toda a entrada.

Os resultados confirmam essa diferença. Para $N = 2.000$, a implementação iterativa apresentou 4.218,75 ns, enquanto a recursiva apresentou 22.184.193,75 ns. A versão recursiva foi aproximadamente 5.258 vezes mais lenta nesse tamanho de entrada. Essa diferença cresce porque o custo quadrático aumenta muito mais rapidamente que o custo linear.

O crescimento entre $N = 100$ e $N = 2.000$ reforça a conclusão: a versão iterativa aumentou cerca de 5,7 vezes, enquanto a recursiva aumentou aproximadamente 411 vezes. Portanto, a curva experimental evidencia o impacto da complexidade $O(N^2)$ sobre entradas maiores.

Em relação à pilha, a implementação iterativa permanece em $O(1)$, enquanto a implementação recursiva apresenta crescimento linear, compatível com $O(N)$.

5.4 Síntese dos resultados

De maneira geral, os resultados experimentais acompanham as complexidades teóricas. P1 apresenta $O(N^2)$ em ambas as versões no tempo, mas a recursiva usa $O(N^2)$ de pilha. P2 apresenta $O(N)$ no tempo em ambas, com $O(1)$ de pilha na iterativa e $O(N)$ na recursiva. P3 apresenta $O(N)$ na iterativa e $O(N^2)$ na recursiva, enquanto a pilha recursiva permanece $O(N)$.

Problema	Implementação	Tempo	Pilha
P1	Iterativa	$O(N^2)$	$O(1)$
P1	Recursiva	$O(N^2)$	$O(N^2)$
P2	Iterativa	$O(N)$	$O(1)$
P2	Recursiva	$O(N)$	$O(N)$
P3	Iterativa	$O(N)$	$O(1)$
P3	Recursiva	$O(N^2)$	$O(N)$

As diferenças entre os valores medidos e uma curva ideal são esperadas. Os tempos dependem do ambiente de execução, do escalonamento do sistema operacional, de outros processos e da resolução do temporizador. Além disso, a medida de pilha utilizada neste trabalho representa a profundidade máxima de chamadas recursivas, e

não uma medição direta em bytes. Por isso, ela deve ser interpretada como uma aproximação do consumo da pilha.

6. CONCLUSÃO

A realização deste trabalho permitiu analisar, de forma prática e teórica, as diferenças entre algoritmos iterativos e recursivos aplicados a três problemas distintos.

Os resultados demonstraram que a escolha da abordagem pode influenciar significativamente o consumo de recursos. No Problema 1, as duas implementações apresentaram complexidade temporal $O(N^2)$, porém a versão recursiva apresentou profundidade de pilha $O(N^2)$. No Problema 2, ambas apresentaram complexidade temporal $O(N)$, enquanto a implementação recursiva apresentou profundidade de pilha $O(N)$. No Problema 3, a diferença temporal foi mais significativa: a implementação iterativa apresentou $O(N)$, enquanto a recursiva apresentou $O(N^2)$, mantendo profundidade de pilha $O(N)$.

Os experimentos foram realizados com 32 execuções para cada combinação de problema, implementação e tamanho de entrada, e a média foi registrada em CSV. Os resultados permitiram observar tendências compatíveis com as análises assintóticas, principalmente no crescimento da pilha de P1 e na diferença temporal entre as versões iterativa e recursiva de P3.

Conclui-se que a análise conjunta de tempo e espaço é fundamental para avaliar uma solução algorítmica. A recursão pode tornar a implementação mais direta, mas pode aumentar a profundidade da pilha e, dependendo da estratégia adotada, elevar significativamente o tempo de execução.

7. ANEXOS

Repositório público do projeto: GitHub - <https://github.com/MatheusSZR/analise-algoritmos-c>

Os códigos-fonte e os arquivos CSV utilizados nos experimentos estão disponíveis no repositório público acima.

ANEXO A – CÓDIGO-FONTE

1. problema1.c
2. problema2.c
3. problema3.c
4. Experimentos.c

ANEXO B – DADOS DOS EXPERIMENTOS

Os dados obtidos nos experimentos foram armazenados em arquivos no formato CSV e utilizados para a geração das tabelas e gráficos apresentados neste relatório.

O arquivo principal contendo os resultados consolidados é:

resultados_consolidados.csv

Também foram geradas tabelas específicas para cada problema:

- p1_iterativo_10.csv a p1_iterativo_50.csv e equivalentes recursivos
- p2_iterativo_*.csv e p2_recursivo_*.csv
- p3_iterativo_*.csv e p3_recursivo_*.csv
- Profundidade da pilha registrada nos mesmos arquivos CSV
-
-

Os arquivos CSV selecionados para a entrega acompanham os códigos-fonte do projeto.

ANEXO C – GRÁFICOS

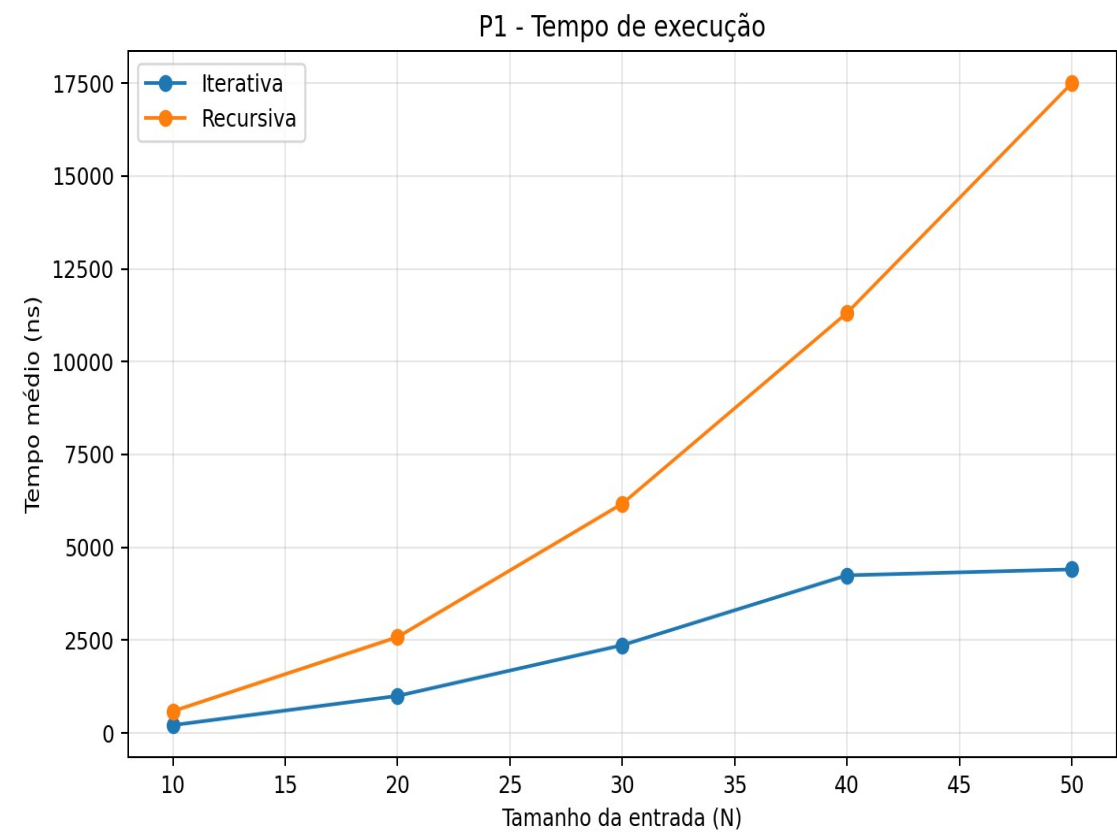


Gráfico C.1 – Problema 1: Tempo de execução

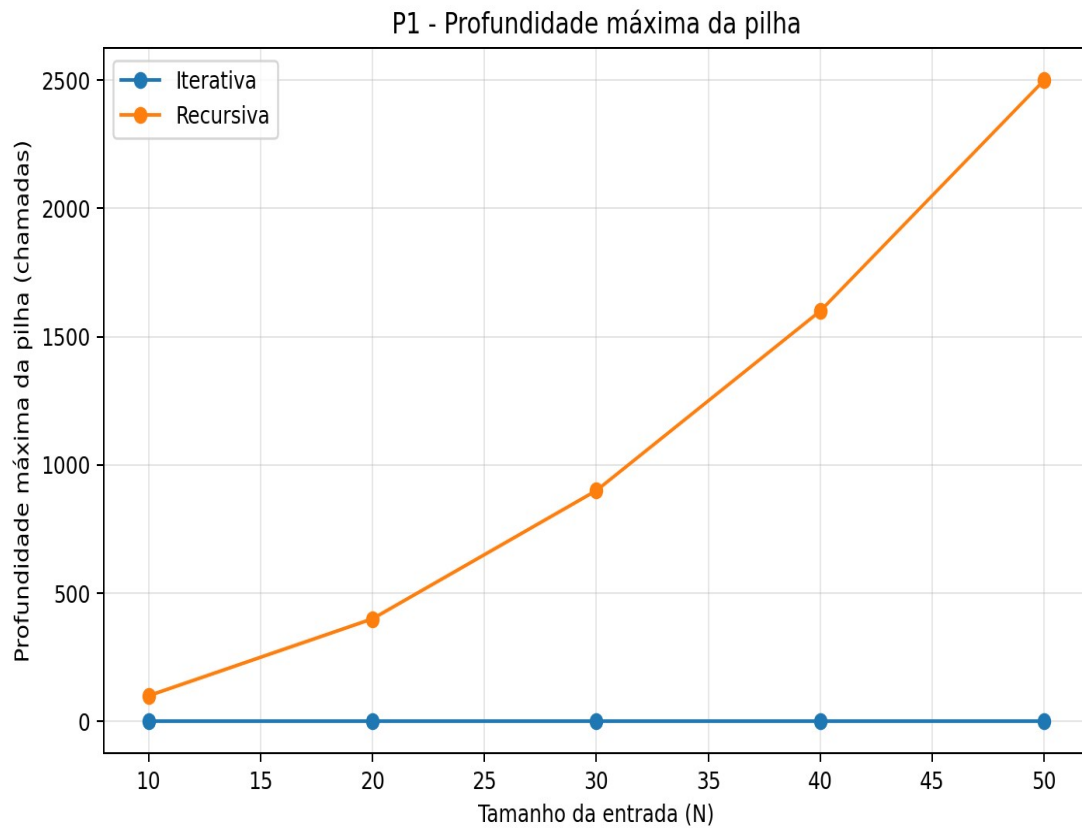


Gráfico C.2 – Problema 1: Consumo de pilha

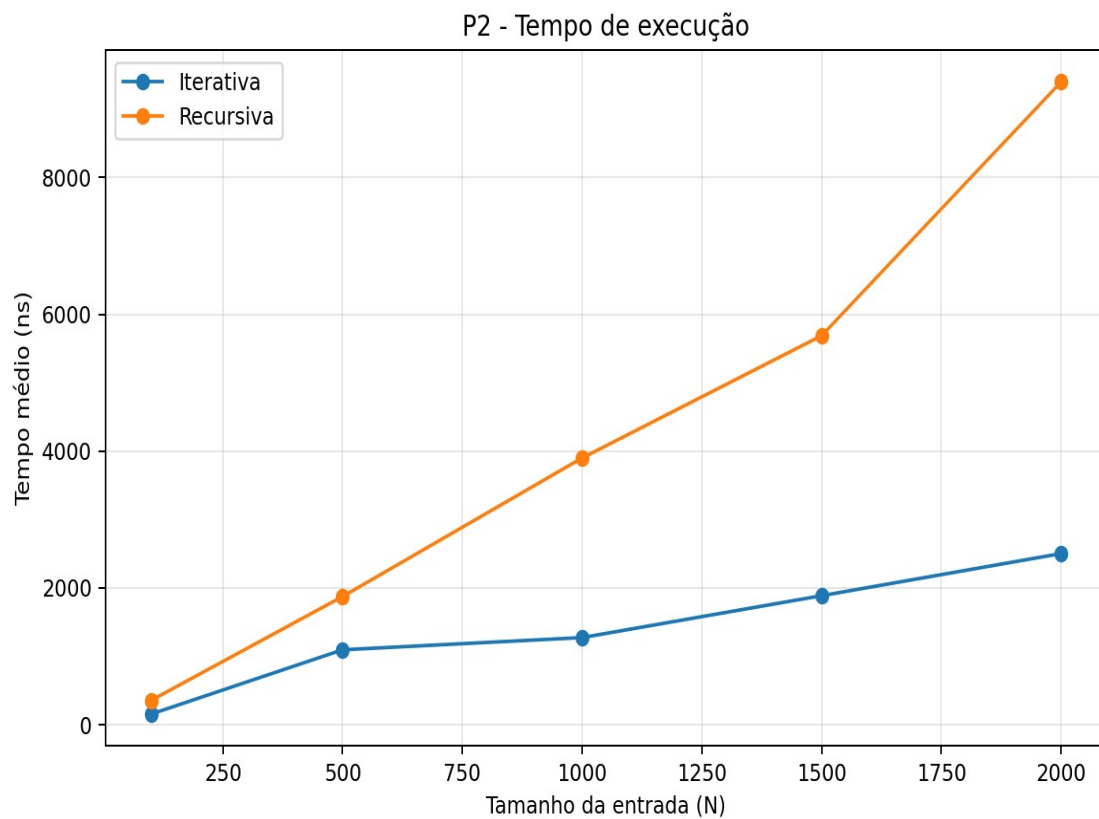


Gráfico C.3 – Problema 2: Tempo de execução

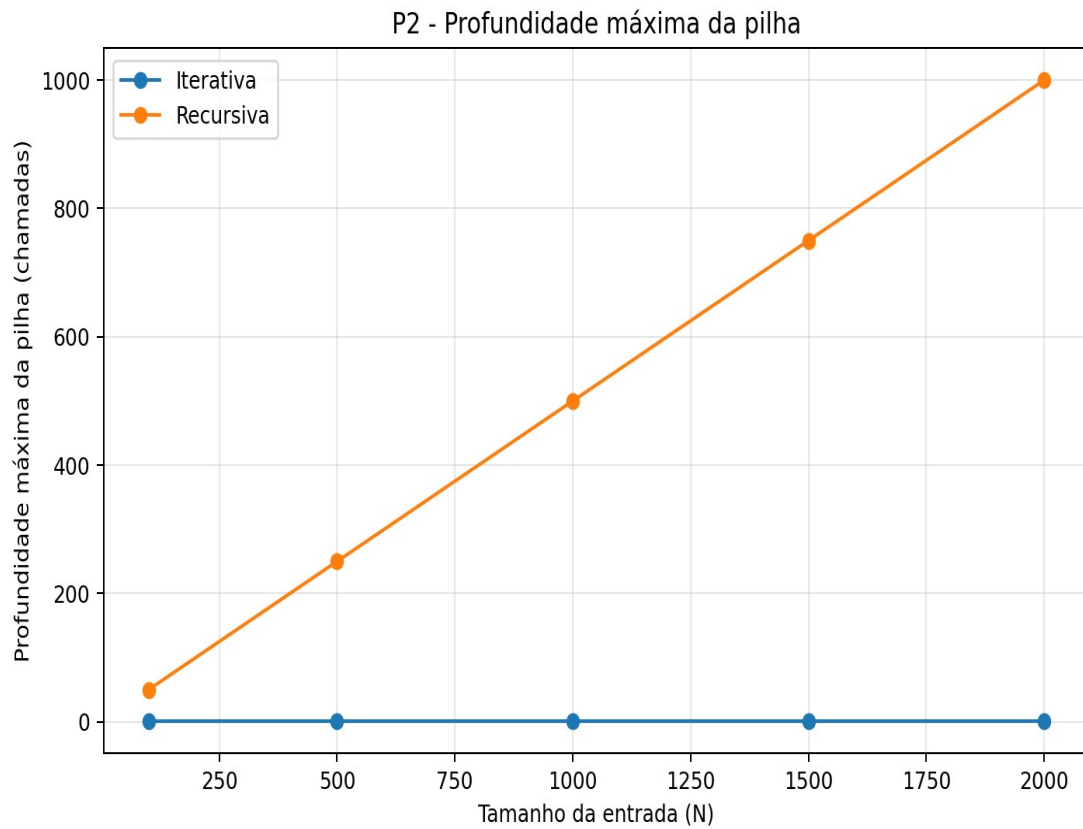


Gráfico C.4 – Problema 2: Consumo de pilha

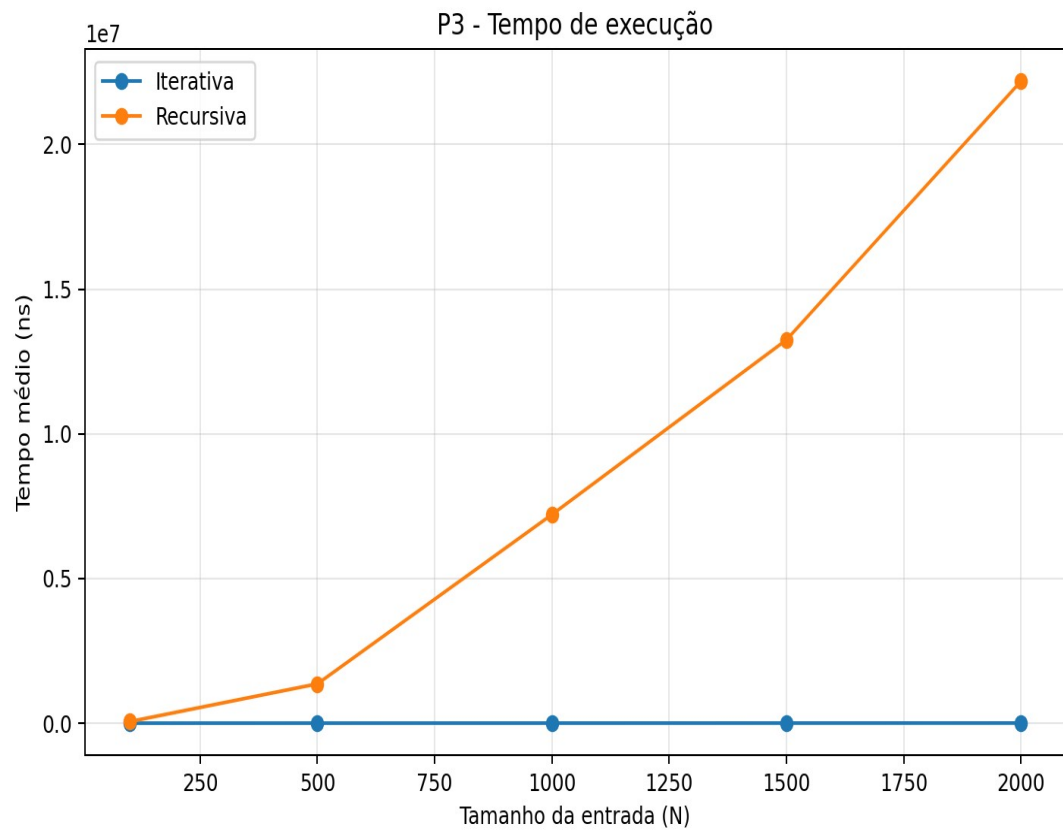


Gráfico C.5 – Problema 3: Tempo de execução

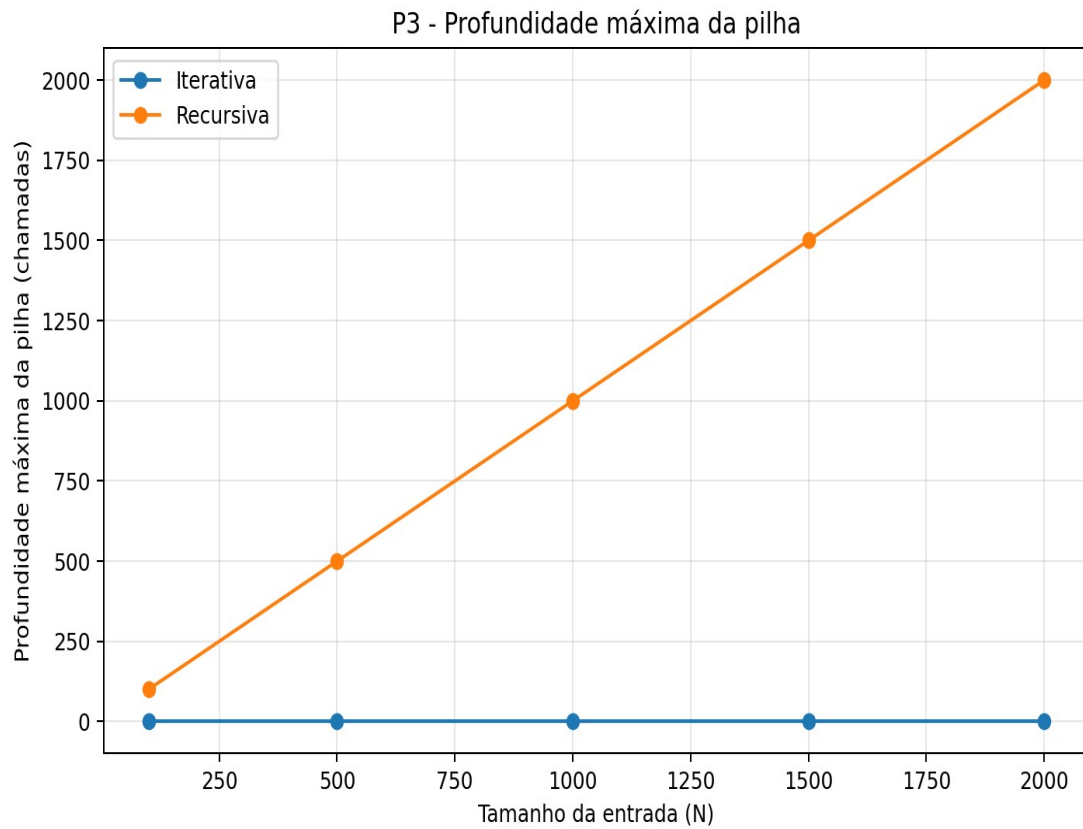


Gráfico C.6 – Problema 3: Consumo de pilha